Proof-of-Concept Implementation of Post-Quantum Secure Messaging utilizing ML-KEM-768

Rupak R. Gupta (231080028) **Ghruank Kothare** (231080038) **Abhay Upadhyay** (231080072)
Third Year Information Technology
Veermata Jijabai Technological Institute (VJTI)
System Security Project Report

April 29, 2026

Abstract

The emergence of practical, fault-tolerant quantum computers actively threatens the foundational security of current digital infrastructures, necessitating an urgent transition to quantum-resistant protocols. Algorithms such as RSA and ECDH can be compromised in polynomial time by sufficient quantum acceleration. This report details a proof-of-concept (PoC) demonstrating a practical, production-quality response to this threat: a terminal-based encrypted messaging system built on the IRC protocol, secured end-to-end by ML-KEM-768 (NIST FIPS 203). We present a C++20-based implementation of a blind-relay chat protocol that combines post-quantum key encapsulation mechanisms with AES-256-GCM symmetric encryption. Empirical outputs from the terminal application confirm the deterministic efficiency of the cryptographic stack, validating that deploying Module-LWE-based cryptosystems is structurally sound, highly performant, and imperative for averting systemic data decryption via "Harvest Now, Decrypt Later" adversarial strategies.

1 Introduction

The rapid progression in quantum processing units (QPUs) mandates an accelerated advancement in global cryptographic protocols. Adversaries are currently engaged in "Harvest Now, Decrypt Later" (HNDL) strategies, recording classically encrypted traffic with the intent to decrypt it once quantum hardware matures. This paradigm shift requires transitioning from reliance on integer factorization and discrete logarithms to multi-dimensional mathematical structures—specifically, lattice-based problems conjectured to be intractable for both classical and quantum computing limits. This report bridges the abstract module-lattice algebra of ML-KEM with tangible software engineering by detailing a proof-of-concept C++20 terminal application, Quantum IRC, establishing its viability for modern secure communications.

2 Objective Framework

The project fulfills a multi-disciplinary technical objective involving algorithm standardization, exact cryptographic integration, and secure application architecture:

- **Demonstrate ML-KEM-768 Application:** Implement the full NIST FIPS 203 KEM handshake inside a working C++20 messaging application, translating theoretical lattice mathematics into a functional proof-of-concept.
- **Achieve End-to-End Post-Quantum Security:** Ensure that messages are confidential against both classical adversaries and future quantum adversaries by pairing ML-KEM-768 with AES-256-GCM and HKDF-SHA256.
- **Design a Blind-Relay Architecture:** Route encrypted packets through a stateless server that never possesses decryption keys, ensuring structural privacy guarantees even under server compromise.

- **Algorithmic Abstraction:** Extract and detail the state-machine operations of ML-KEM and map the abstract linear algebra of Module-LWE into an observable 3D topological environment.

3 Theoretical Background

3.1 The Quantum Threat Context

Traditional asymmetric cryptography relies on the difficulty of abelian-group problems. Shor's algorithm achieves an exponential reduction in time complexity for integer factorization, reducing the problem to finding the period r of the modular function $f(x) = a^x \pmod{N}$. The quantum superiority lies in executing the modular exponentiation over a superposition of all possible inputs simultaneously, followed by the Quantum Fourier Transform (QFT), which induces constructive interference precisely at multiples of the inverse period $1/r$. This completely breaks RSA and standard Elliptic Curve Cryptography (ECC).

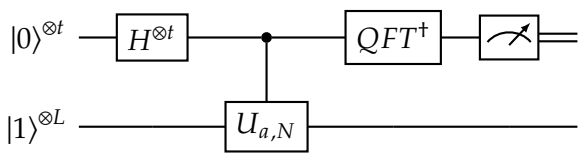

Figure 1: Core Architecture of Shor's Algorithm. The QFT resolves the periodic phase induced by the modular exponentiation oracle, breaking RSA in polynomial time.

Concurrently, Grover's algorithm provides a quadratic speedup for unstructured search operations, operating in $O(\sqrt{N})$ time compared to classical $O(N)$. It sequentially applies an oracle operator $U_\omega = I - 2|\omega\rangle\langle\omega|$ and a diffusion operator $U_s = 2|s\rangle\langle s| - I$. This theoretically degrades the effective security of symmetric standards like AES-256 to AES-128, necessitating the adoption of larger key sizes for symmetric primitives.

3.2 Module-LWE and ML-KEM Mechanics

NIST FIPS 203 standardizes ML-KEM based on the Module Learning With Errors (M-LWE) hypothesis. Security rests on the hardness of recovering a secret $\mathbf{s}$ from the equation $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$ due to the discrete, bounded error term $\mathbf{e}$.

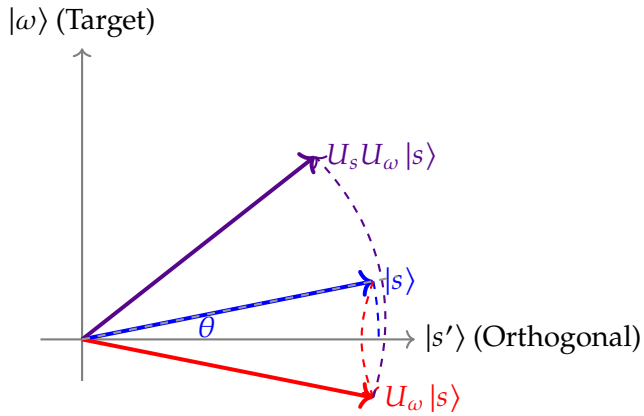

Figure 2: 2D Subspace Representation of Amplitude Amplification in Grover's algorithm. Each iteration rotates the state vector toward the target state, halving the effective bit-strength of symmetric keys.

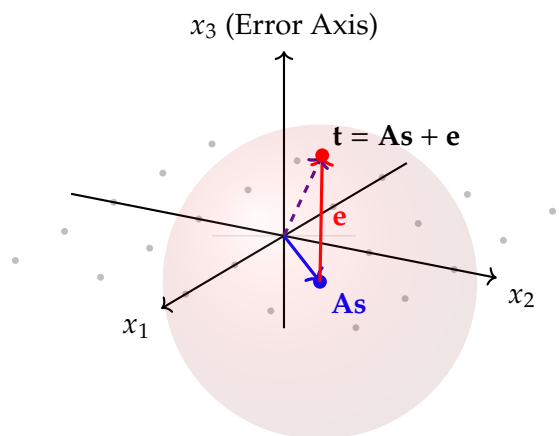

Figure 3: 3D Geometric Visualization of M-LWE. Finding the underlying plane generated by $\mathbf{A}$ from the set of spatial coordinates defined by $\mathbf{t}$ becomes computationally intractable due to the 3D translation vector $\mathbf{e}$.

The ML-KEM-768 parameters (NIST Level 3) utilize a module dimension $k = 3$ and modulus $q = 3329$, over the polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^{256} + 1)$. The encapsulation procedure ensures establishing a secure shared secret across an insecure channel.

Algorithm 1: ML-KEM Encapsulation

Input: Public Key $pk = (\mathbf{A}, \mathbf{t})$
Output: Ciphertext $c = (\mathbf{u}, \mathbf{v})$, Shared Secret K

$m \leftarrow \mathcal{U}(\{0,1\}^{256})$ `// Internal Secret`
$r \leftarrow$ Randomness Oracle
$\mathbf{r}, \mathbf{e}_1, e_2 \leftarrow \text{SampleCBD}_\eta(r)$
$\mathbf{u} \leftarrow \mathbf{A}^T\mathbf{r} + \mathbf{e}_1 \pmod q$
$\mathbf{v} \leftarrow \mathbf{t}^T\mathbf{r} + e_2 + \text{Encode}(m) \pmod q$
$c \leftarrow \text{Compress}(\mathbf{u}, \mathbf{v})$
$K \leftarrow \text{KDF}(m \parallel H(c))$
return c, K

3.3 Hybrid Cryptographic Stack Integration

To utilize the 32-byte ML-KEM shared secret operationally, HKDF-SHA256 (RFC 5869) is applied to derive a uniform 256-bit AES session key. Every subsequent message is encrypted with AES-256-GCM, utilizing a fresh 12-byte nonce generated via a cryptographically secure pseudo-random number generator (CSPRNG) to guarantee authenticated encryption and payload integrity.

4 C++ Cryptographic Implementation

The proof-of-concept infrastructure is explicitly designed for POSIX environments. The implementation is decoupled into a core cryptographic library and a networked messaging layer. This section details the standalone cryptographic logic.

4.1 Core Architecture

C++20 Cryptographic Client: The terminal application utilizes modern constructs (`std::span`, structured bindings). A self-contained FIPS 203 library handles post-quantum operations natively, while OpenSSL 3.0+ is strictly scoped to handle AES-256-GCM and HKDF procedures.

4.2 Performance Benchmarks

The benchmark evaluated the ML-KEM suite over 1000 iterations per parameter set, confirming functionality and measuring execution latency.

```
★ ML-KEM Performance Benchmark (1000 iterations each)

Parameter Set | KeyGen          | Encaps          | Decaps          |
ML-KEM-512    | KeyGen:  274 us | Encaps:  256 us | Decaps:  403 us | OK
ML-KEM-768    | KeyGen:  383 us | Encaps:  426 us | Decaps:  644 us | OK
ML-KEM-1024   | KeyGen:  561 us | Encaps:  632 us | Decaps:  955 us | OK
```

Figure 4: ML-KEM Performance Benchmark (1000 iterations each).

These metrics demonstrate the expected computational scaling. The `OK` flag confirms that encapsulated and decapsulated shared secrets matched correctly across all iterations.

4.3 Standalone Execution Flow

An interactive standalone demo validated the hybrid cryptographic flow (ML-KEM + AES-256-GCM) prior to the networking layer integration.

```
★ === ML-KEM-768 Secure Messaging Demo (FIPS 203) ===

Who is the sender?  Who is the receiver?

[ML-KEM] Key Generation complete.
[Bob] Publishes public key.

[Sender] Encapsulating shared secret...
[Sender] Encapsulated Key (hex):  3680880a9eb301de7daf6a3b5cef6265...

[AEAD] Encrypting message:  "Post-quantum secure messaging"
[AEAD] Ciphertext (hex):   85fd8dc9fd4398c01e02f60af0efcc31...
[AEAD] Auth Tag (hex):     f31a8c2eb961b522684c92c9641eb7d9
[AEAD] Nonce (hex):        2331df99bfaa6d26ae9cdc73

[Receiver] Decapsulating shared secret...
[Session] E2E secure channel established.

=== RESULTS ===
Bob received:  "Post-quantum secure messaging"
(Message integrity:  VERIFIED by AES-256-GCM authentication)
```

Figure 5: Local Secure Messaging Flow.

5 Blind-Relay IRC Integration

With the core cryptographic logic validated, the system routes packets across an insecure TCP network.

5.1 Network Architecture

Blind-Relay IRC Protocol: The server infrastructure acts exclusively as a stateless router

handling user authentication via PBKDF2-SHA256 hashed credentials, and forwards opaque hex-encoded AES-GCM ciphertexts between clients.

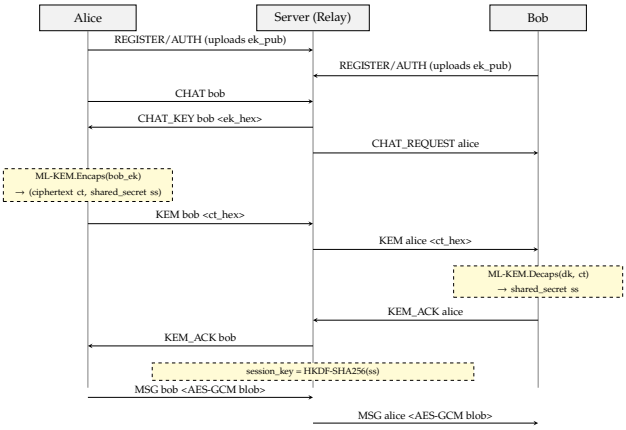

Figure 6: ML-KEM-768 Key Exchange Sequence Flow.

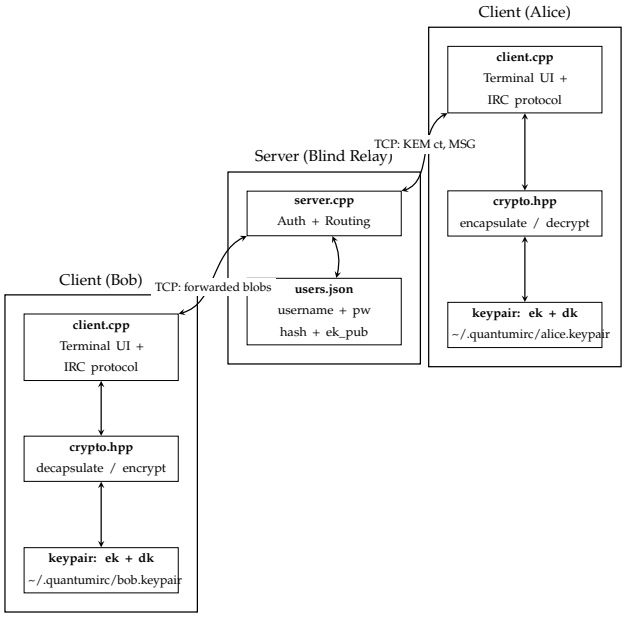

Figure 7: Component Architecture displaying endpoints and the stateless relay.

5.2 Protocol Execution Outputs

5.2.1 User Registration and Keypair Generation

On first launch, the client generates an ML-KEM-768 keypair locally. The 2400-byte decapsulation key is retained securely on disk, and the 1184-byte encapsulation key is uploaded to the relay server.

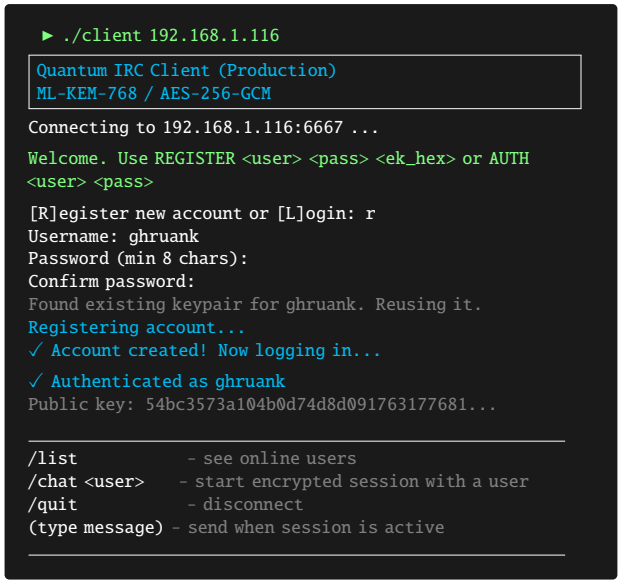

Figure 8: Client registration: keypair generation and server upload.

5.2.2 KEM Handshake and Session Establishment

To initiate communication, the client fetches the target's public encapsulation key from the server, executes `ML-KEM.Encaps(ek)`, and sends the 1088-byte ciphertext to the peer. The peer runs `ML-KEM.Decaps(dk, ct)` to recover the exact same shared secret.

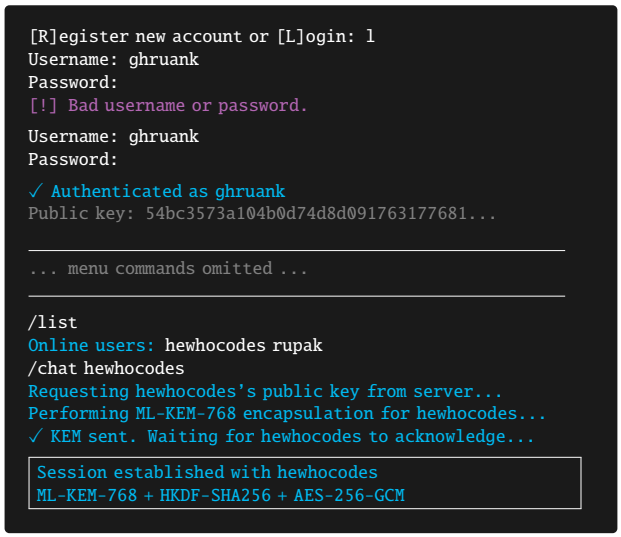

Figure 9: Login, user discovery, and ML-KEM-768 handshake.

5.2.3 End-to-End Encrypted Exchange

Once established, the payload is verified. The terminal displays both the raw hex ciphertext transmitted over the TCP socket and the decrypted plaintext locally, confirming zero-knowledge routing by the server.

```
Session established with rupakgupta
ML-KEM-768 + HKDF-SHA256 + AES-256-GCM
[you] encrypted : 6ef7275020b5e83400ce9489cdc...
[you] decrypted : hello rupak
you:
[rupakgupta] encrypted : d710857650dd0f339d58a83...
[rupakgupta] decrypted : hello ghruank from ty it
[you] encrypted : 3090d721f88484766d69dea35c3...
[you] decrypted : how are you
you:
[rupakgupta] encrypted : c622b23ede85efe655d04a0...
[rupakgupta] decrypted : im fine thank you
```

```
★ ghruankkothare is starting a secure session...
Performing ML-KEM-768 decapsulation (KEM)...
Session established with ghruankkothare
ML-KEM-768 + HKDF-SHA256 + AES-256-GCM
you:
[ghruankkothare] encrypted : 6ef7275020b5e83400ce9...
[ghruankkothare] decrypted : hello rupak
[you] encrypted : d710857650dd0f339d58a83...
[you] decrypted : hello ghruank from ty it
you:
[ghruankkothare] encrypted : 3090d721f88484766d6...
[ghruankkothare] decrypted : how are you
[you] encrypted : c622b23ede85efe655d04a0...
[you] decrypted : im fine thank you
```

Figure 10: E2E exchange: ciphertext transmitted, plaintext localized.

This output confirms all three structural properties: Confidentiality (only hex traverses the network), Integrity (AES-GCM tags reject tampering), and Post-Quantum Resistance (shared secret established without abelian reliance).

6 Real-world Deployment Impact

The successful execution of this proof-of-concept addresses key transitional hurdles for the impending global PQC migration:

- **FIPS 203 Integration Pathways:** Validates the exact software architectural parameters required for enterprise messaging platforms to replace ECDH algorithms with ML-KEM without disrupting user experience.
- **Threat Modeling Re-evaluation:** Adjusts symmetric key lifecycles to accommodate Grover's square-root acceleration by enforcing AES-256.
- **Stateless Resilience:** Proves that practical post-quantum secure messaging does not require trusting the transport infrastructure; the blind-relay natively supports private network deployments.

7 Project Availability

The theoretical visualization suite and C++20 Quantum IRC implementation codebase are decoupled and independently verifiable:

- **Web Deployment:** https://kyber-pqc.vercel.app
- **Quantum IRC Repository:** https://github.com/ghruank/irc-encrypted

8 Conclusion

This project successfully demonstrates a functional, production-aligned proof-of-concept for post-quantum encrypted messaging utilizing ML-KEM-768. By coupling a NIST-standardized key encapsulation mechanism with classical AEAD symmetric cryptography within a blind-relay architecture, the system provides end-to-end confidentiality secure against both current classical adversaries and anticipated future quantum decryption capabilities. The empirical execution validates that deploying lattice-based cryptosystems is structurally viable, performant, and ready for immediate infrastructural integration.

References

[1] National Institute of Standards and Technology (NIST), "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," August 2024. https://doi.org/10.6028/NIST.FIPS.203

[2] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," *Proceedings 35th Annual Symposium on Foundations of Computer Science*, IEEE, 1994, pp. 124-134.

[3] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proceedings*

of the twenty-eighth annual ACM symposium on Theory of computing, 1996, pp. 212-219.

[4] H. Krawczyk, P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, May 2010.

[5] NIST, "SP 800-227 (IPD): Recommendations for Key-Encapsulation Mechanisms," 2024. https://csrc.nist.gov/pubs/sp/800/227/ipd